INDUSTRIAL INTERNSHIP REPORT

Smart City Traffic Forecasting

Machine Learning based Traffic Forecasting, Congestion Monitoring and Anomaly Detection

Prepared by

Harsh Kumar

Lakshmi Narain College of Technology - Bhopal

EXECUTIVE SUMMARY

Project Summary

This report documents the development of Smart City Traffic Forecasting, an interactive machine-learning dashboard designed to analyze hourly traffic observations and generate short-term junction-level forecasts. The application accepts a built-in demonstration dataset or a user-uploaded CSV file, validates the required traffic columns, engineers temporal, lag and rolling features, trains two tree-based regression models, evaluates their performance and selects the model with the lower mean absolute error. The selected model is then used to generate 6-, 12-, 24- or 48-hour traffic forecasts. The system additionally converts predicted demand into speed and congestion indicators, detects unusual traffic records with an interquartile-range method, visualizes the results with interactive Plotly charts and exports forecast outputs to TXT, CSV and Excel formats. The implementation uses Python, Pandas, NumPy, Scikit-learn, Plotly and Streamlit. In the captured demonstration run for Ayodhya Bypass, the dashboard displayed Gradient Boosting Regressor as the selected model with MAE 48.72, RMSE 62.60 and R^2 0.9662; the interface also reported a forecast peak of 1,757 vehicles/hour and an average forecast speed of 23.6 km/h. These values represent the shown demonstration run and are included as observed application outputs rather than as a universal benchmark.

TABLE OF CONTENTS

Section	Title	Page	Section	Title	Page
1	Preface	3	5.3	Feature Engineering Pipeline	16
2	Introduction	4	5.4	Model Training and Selection	17
2.1	About UniConverge Technologies Pvt. Ltd.	4	5.5	Forecast Generation	18
2.2	About upSkill Campus	8	5.6	Interfaces and Dashboard Design	19
2.3	The IoT Academy	9	5.7	Data Schema and Processing Rules	19
2.4	Objectives of the Internship Program	9	6	Performance Test	20
2.5	References	9	6.1	Constraints and Test Objectives	20
2.6	Glossary	9	6.2	Test Plan / Test Cases	21
3	Problem Statement	10	6.3	Test Procedure	22
3.1	Background and Motivation	10	6.4	Performance Outcome	23
3.2	Problem Definition	10	6.5	Observed Demonstration Results	24
3.3	Functional and Non-Functional Requirements	11	7	My Learnings	25
4	Existing and Proposed Solution	12	7.1	Technical Learning	25
4.1	Existing Approaches and Limitations	12	7.2	Analytical and Professional Learning	26
4.2	Proposed Solution	12	8	Future Work Scope	27
4.3	Value Addition	13	8.1	Planned Enhancements	27
4.4	Submission Placeholders	13	8.2	Limitations of Current Implementation	28
5	Proposed Design / Model	14	Appendix A	Project Outcome Screenshots	29
5.1	High Level System Architecture	14	Appendix B	Selected Code Components	32
5.2	Low Level Data Flow	15	Conclusion	Conclusion and References	34

1 Preface

The industrial internship was used as an opportunity to connect classroom learning in machine learning and data science with an applied urban-transportation problem. The assigned project, Smart City Traffic Forecasting, required the development of a working analytical solution rather than a purely theoretical model. The final application therefore combines data preparation, feature engineering, predictive modeling, visualization, monitoring logic and report generation in a single Streamlit interface.

Summary of the Work

The work progressed from understanding the traffic forecasting problem and inspecting the structure of time-based data to implementing a complete prediction pipeline. The application supports both demonstration data and user-uploaded CSV data. After validation, the data is sorted by junction and timestamp, temporal features are created, and historical traffic values are transformed into lag and rolling-window predictors. Random Forest and Gradient Boosting regression models are trained and compared. The lower-MAE model is used for the forecasting stage, after which the system produces short-term traffic, speed and congestion views.

Need for Relevant Internship Experience

Traffic forecasting is a practical data-science problem because decision makers need information before demand peaks occur. An internship environment creates space to practice requirements analysis, data handling, model selection, debugging, interface design and documentation as a connected workflow. It also develops the discipline of validating assumptions and reporting measurable outcomes.

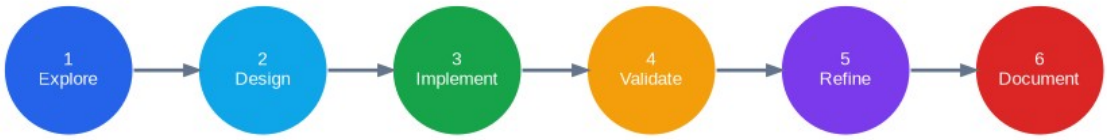


Figure 1.1: Internship work flow used for the Smart City Traffic Forecasting project

Overall experience: the internship strengthened practical confidence in Python-based analytics and improved the ability to explain how raw traffic observations become model inputs, predictions and decision-support information.

2 Introduction

2.1 About UniConverge Technologies Pvt. Ltd.

The provided sample report describes UniConverge Technologies Pvt. Ltd. (UCT) as an organization established in 2013 and working in the digital-transformation domain with an industrial focus on sustainability and return on investment. Its technology landscape includes Internet of Things, cyber security, cloud computing, machine learning, communication technologies such as 4G/5G/LoRaWAN, Java full stack, Python and front-end technologies. These areas form a useful industrial context for a data-driven traffic application because smart-city systems depend on data collection, connectivity, analytics and operational visualization.



Figure 2.1: UCT overview shown in the provided internship sample

For this report, the organization context is relevant because the project uses a software-centric analytics workflow that can naturally connect to broader smart-city and IoT data systems. The present implementation, however, is a local Streamlit application and does not directly integrate live IoT devices or city traffic APIs.

UCT Technology Context

- Internet of Things and industrial data connectivity provide a foundation for continuous measurement and monitoring.
- Cloud computing can support scalable deployment, data storage and shared analytics services.
- Machine learning can convert historical measurements into predictive information for planning and operations.

2.1.1 UCT Insight / IoT Platform Context

The sample report introduces UCT Insight as an IoT platform designed for rapid deployment of IoT applications and business/process insight. It describes a Java backend, ReactJS front end, support for MySQL and NoSQL databases, device connectivity through MQTT, CoAP, HTTP, Modbus TCP and OPC UA, and support for cloud as well as on-premises deployments. The same sample lists dashboard building, analytics and reporting, alerts and notifications, third-party integration and a rule engine among the platform features.

Figure 2.2: UCT Insight branding from the sample report

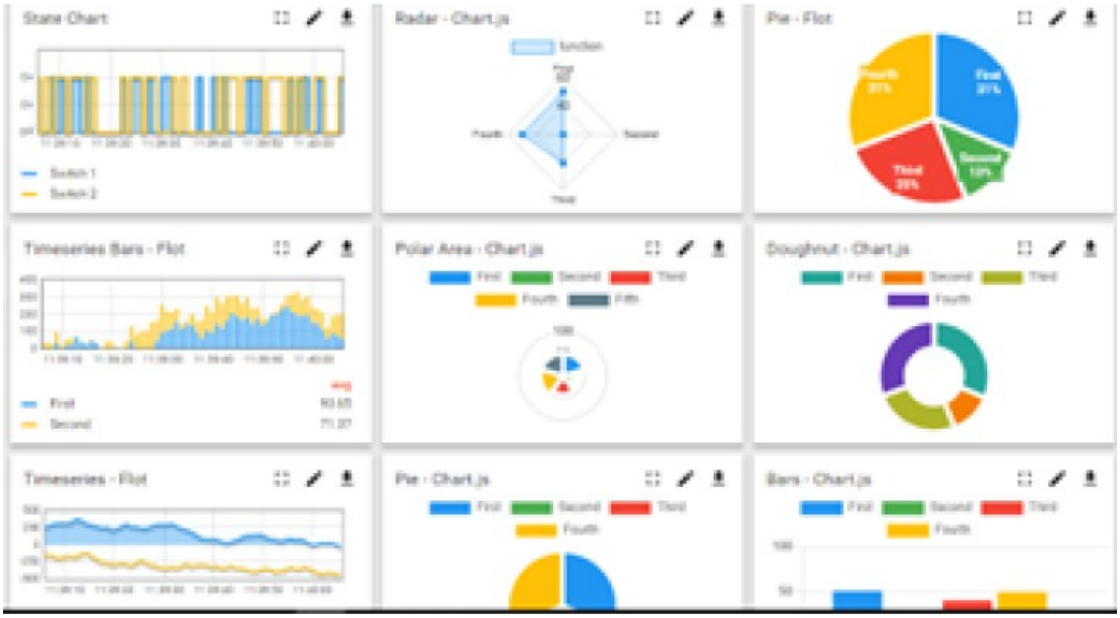


Figure 2.3: Example analytical dashboard imagery from the sample report

The traffic forecasting project fits conceptually within this type of analytics environment: traffic observations can be collected, transformed into features, modeled and surfaced through dashboards. The current project focuses specifically on the machine-learning and visualization layer and uses Streamlit for the interface.

2.1.2 Smart Factory and Predictive-Monitoring Context

The sample report also presents Factory Watch as a smart-factory platform supporting production and asset monitoring, OEE, predictive maintenance, modular services and scalable SaaS delivery. Although the present project is transportation-oriented, the same design principle is visible: operational data becomes useful when it is organized into metrics, trends, forecasts and decision-support views.

Figure 2.4: Factory Watch branding from the sample report



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

Figure 2.5: Example factory monitoring dashboard from the sample report

2.1.2.1 Industrial Monitoring Relevance



Figure 2.6: Example 3D smart-factory monitoring visualization from the sample report

This contextual comparison helped shape the project interface around operator-friendly outputs such as KPI cards, traffic trends, congestion labels, feature importance and exportable reports instead of presenting only raw model values.

Industrial Relevance to the Project

Industrial Pattern	How It Appears in This Project
Data collection and connectivity	CSV input acts as the current data-ingestion interface and can later be replaced or supplemented by live feeds.
Monitoring and visualization	Streamlit presents current demand, forecasts, speed, congestion and anomaly information.
Predictive analytics	Tree-based regression converts historical demand patterns into future traffic estimates.
Operational insight	Peak-period and congestion summaries turn model output into information that a traffic operator can review.

2.1.3 LoRaWAN-Based Solutions and Predictive Maintenance

The sample report describes UCT as an early adopter of LoRaWAN technology for solutions including agriculture, smart cities, industrial monitoring, smart street lighting and smart metering. It also describes predictive maintenance based on embedded systems, industrial IoT and machine learning. These examples illustrate the broader industrial use of data streams, monitoring, forecasting and anomaly identification.

Figure 2.7: LoRaWAN technology reference in the sample

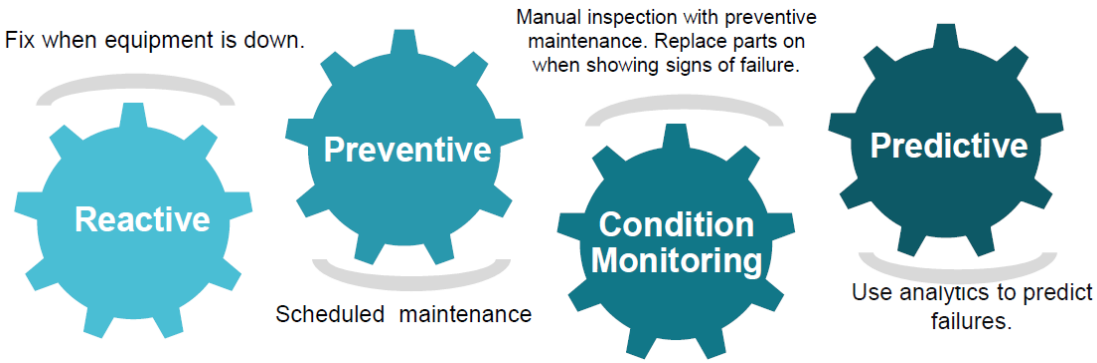


Figure 2.8: Predictive-maintenance lifecycle reference in the sample

2.2 About upSkill Campus

The sample report describes upSkill Campus as a career-development platform that supports personalized and measurable career development. Within the internship model, the platform and The IoT Academy facilitate the learning and project process while the industrial partner provides the applied problem context.

2.3 The IoT Academy

The sample report identifies The IoT Academy as the EdTech division of UCT and notes collaboration with EICT Academy and IIT academic ecosystems for executive certification programs. This context is included here to maintain consistency with the provided report format.

2.4 Objectives of the Internship Program

- Gain practical experience in applying machine-learning concepts to an industry-oriented problem.
- Translate a problem statement into a working data-processing and forecasting workflow.
- Practice data validation, feature engineering, model training and quantitative evaluation.
- Build an interactive dashboard that communicates results to a non-technical user.
- Improve documentation, analytical reasoning, debugging and professional communication.

2.5 References

1. Scikit-learn documentation for RandomForestRegressor, GradientBoostingRegressor and regression metrics.
2. Streamlit documentation for interactive Python application development.
3. Plotly Python documentation for interactive charting.
4. Pandas and NumPy documentation for data manipulation and numerical computing.
5. Project source code and application screenshots supplied by the author for this report.

2.6 Glossary

Term	Acronym / Meaning
Machine Learning	ML - learning patterns from data to make predictions or decisions
Mean Absolute Error	MAE - average absolute prediction error
Root Mean Squared Error	RMSE - square root of mean squared prediction error
Coefficient of Determination	R^2 - proportion of variance explained by the model, relative to the test baseline
Interquartile Range	IQR - Q3 minus Q1; used here for anomaly limits
CSV	Comma-Separated Values data file
Lag Feature	Previous observed traffic value used as a predictor
Rolling Mean	Average over a recent historical window
Congestion	Traffic-load classification based on demand-to-capacity ratio
Streamlit	Python framework used for the dashboard interface
Random Forest	Ensemble of decision trees used for regression
Gradient Boosting	Sequential ensemble method used here for regression
EDA	Exploratory Data Analysis
KPI	Key Performance Indicator

3 Problem Statement

3.1 Background and Motivation

Urban traffic demand changes over the day and is influenced by temporal routines, location, weather and special events. A dashboard that only reports what has already happened is useful for monitoring, but it cannot directly answer the operational question of what demand may occur over the next few hours. For traffic staff, planning ahead can be more useful than reacting after a peak has already formed.

The project therefore addresses a forecasting problem: given historical hourly traffic records for multiple junctions and a set of contextual variables, estimate future traffic volume and convert that estimate into interpretable speed and congestion indicators. The implementation is intended as a practical prototype that can be demonstrated, inspected and extended.

3.2 Problem Definition

Problem Definition

Design and implement a user-facing machine-learning application that can ingest traffic observations, learn historical demand patterns, generate short-term junction-level traffic forecasts, identify potentially unusual traffic records, classify congestion and present the results through an interactive dashboard.

3.3 Stakeholders / Intended Users

- Traffic monitoring teams needing a quick view of current and near-future demand.
- Data-science trainees who need a reproducible end-to-end forecasting workflow.
- Project reviewers evaluating model metrics, feature importance and visible outputs.
- Future smart-city systems that could consume the forecasting layer as one component.

3.3 Functional and Non-Functional Requirements

Requirement Type	Requirement	Implementation Evidence
Functional	Accept traffic CSV input	File uploader plus column normalization and validation
Functional	Generate traffic forecasts	6/12/24/48 hour horizon controls
Functional	Use multiple ML models	Random Forest + Gradient Boosting training
Functional	Select model objectively	Lower MAE model is chosen
Functional	Classify congestion	Low/Medium/High based on demand-to-capacity ratio
Functional	Detect unusual records	IQR-based anomaly flag
Functional	Present analytics	Plotly charts, metrics, tables and traffic intelligence
Functional	Export results	TXT report, CSV forecast and Excel workbook
Non-functional	Usable interface	Wide Streamlit layout, sidebar controls and clear KPI cards
Non-functional	Robust data handling	Datetime parsing, numeric coercion and invalid-row removal
Non-functional	Reproducibility	Random state fixed at 42 for generated data and models
Non-functional	Extensibility	CSV schema accepts optional speed, rain, temperature and event columns

Design Constraints

- The built-in demonstration dataset is synthetic and should not be treated as a substitute for calibrated live city-sensor data.
- Forecasting uses the latest available temperature, rain and event values when generating future rows; live future covariates are not fetched automatically.
- The current application is a dashboard prototype and does not control physical traffic signals or road infrastructure.
- The code trains models at application runtime rather than using persisted model files.

4 Existing and Proposed Solution

4.1 Existing Approaches and Limitations

For a practical comparison, traffic monitoring can be organized around several baseline patterns. Historical dashboards summarize previously observed demand. Rule-based systems can label congestion when a measured value crosses a predefined threshold. Standalone forecasting scripts can produce predictions but may not provide an integrated monitoring interface. Each approach can be useful, but a fragmented workflow increases the effort required to move from data ingestion to forecast interpretation.

Approach	Strength	Limitation for this Project
Historical charts / spreadsheets	Easy to inspect previous traffic behavior	No built-in forward forecast; limited automation
Threshold-only monitoring	Simple and understandable congestion rules	Does not estimate future demand
Single-model forecasting script	Produces numeric predictions	May lack model comparison and operator-facing dashboard
Integrated ML dashboard (proposed)	Forecast + metrics + congestion + anomalies + export in one workflow	Needs better live-data integration for production use

4.2 Proposed Solution

The proposed application integrates data ingestion, preprocessing, feature engineering, model training, evaluation, forecasting and presentation in a single interactive workflow. It can operate on a built-in 120-day hourly demonstration dataset or a user-supplied CSV file. The model layer contains Random Forest and Gradient Boosting regressors. After evaluation, the lower-MAE model is used for the forecast engine. Forecasted demand is translated into estimated speed for configured junctions and into congestion classes using capacity ratios.

4.3 Value Addition

- Multi-model comparison rather than committing to one regression algorithm from the start.
- Time-aware predictors using lag and rolling statistics to capture recent traffic demand.
- Short-horizon forecasting options ranging from 6 to 48 hours.
- Operator-oriented traffic intelligence: peak period, expected demand, congestion and planning note.
- Junction comparison to view current demand, forecast peak and average speed side by side.
- IQR anomaly detection to highlight unusually high or low traffic records.
- One-click result export to TXT, CSV and Excel.

4.4 Submission Links

Code Submission (GitHub)

https://github.com/HarshKumar7488/upskillcampus/blob/b2e3a487d74dcc6694e900d5cd8462779d6ac5ad/SmartCityTrafficForecasting.py

Report Submission (GitHub)

https://github.com/HarshKumar7488/upskillcampus/blob/b2e3a487d74dcc6694e900d5cd8462779d6ac5ad/SmartCityTrafficForecasting_Harsh_USC_UCT.pdf

5 Proposed Design / Model

5.1 High Level System Architecture

The architecture follows a simple pipeline: traffic data enters the system, is cleaned and transformed into features, is used to train or apply the selected regression model, and then passes through a decision layer that produces traffic forecasts, speed estimates, congestion labels and anomaly information. The final outputs are exposed through a Streamlit dashboard and export functions.

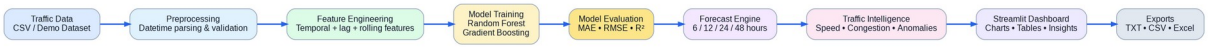


Figure 5.1: High-level system architecture of the Smart City Traffic Forecasting solution

The design intentionally keeps the application modular. Data preparation is separated from feature engineering, model training, forecast generation, decision rules, visualization and reporting. This makes individual components easier to test and replace.

5.2 Low Level Data Flow

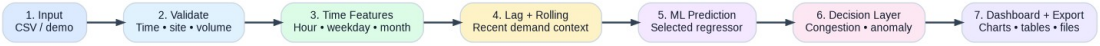


Figure 5.2: Low-level data flow from input data to dashboard outputs

The upload pathway recognizes several possible names for the timestamp and traffic columns. Timestamp candidates include timestamp, datetime, date_time, date and time. Traffic candidates include vehicles, traffic, volume, vehicle_count and traffic_volume. If a junction field is not present, the code assigns the label “Uploaded Junction”.

Optional fields for speed, rain, temperature and event are accepted when present. Missing optional values are supplied with neutral defaults so that the forecasting interface remains usable. Invalid timestamps and non-numeric traffic values are removed before modeling.

5.3 Feature Engineering Pipeline



Figure 5.3: Feature engineering stages implemented in the project

The final feature vector contains fifteen predictors. Four are calendar/time descriptors, four are lag features, three are rolling means, and four are environmental/event variables. The lag features are created within each junction so that one location does not borrow demand history from another.

Feature Group	Features	Purpose
Temporal	hour, day_of_week, month, is_weekend	Capture recurring calendar patterns
Lagged demand	lag_1, lag_2, lag_3, lag_24	Represent recent and day-level history
Rolling context	rolling_3, rolling_6, rolling_24	Smooth short-term demand fluctuations
Contextual	temperature, rain, event	Represent external conditions available in the data

Why these features matter

The most direct information available before a future traffic hour is the history of recent traffic at the same junction. The lag variables preserve this history explicitly, while rolling means reduce the effect of short-lived fluctuations. Calendar variables capture regular commuter patterns, and contextual variables provide additional information when the corresponding measurements are present in the dataset.

5.4 Model Training and Selection

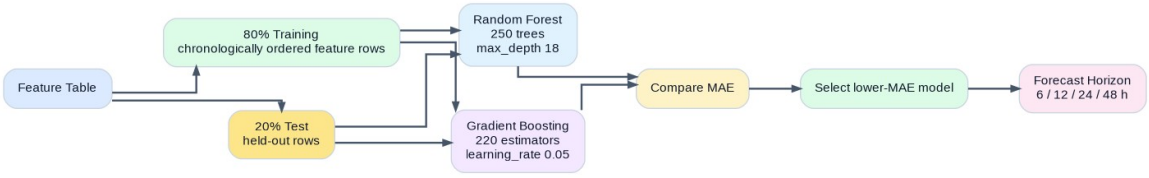


Figure 5.4: Model training, comparison and selection flow

Random Forest Regressor

The implementation uses 250 trees, a maximum depth of 18, minimum samples per leaf of 2, random state 42 and all available processor threads. Random Forest is useful here because it can model nonlinear relationships in structured tabular features and can provide feature-importance values.

Gradient Boosting Regressor

The implementation uses 220 estimators, a learning rate of 0.05, maximum depth 4, Huber loss and random state 42. The lower-MAE model from the two candidates becomes the selected production model for the current application run.

Evaluation is performed using MAE, RMSE and R^2. The current code first trains both models, calculates their test-set MAE, selects the lower-MAE model, and then calculates the final RMSE, R^2 and MAE on the selected predictions.

5.5 Forecast Generation

For a selected junction, the forecast engine takes the latest traffic history and predicts one future hour at a time. At each step, it builds the temporal variables for the new timestamp, carries forward the latest available contextual variables and calculates lag/rolling statistics from the progressively extended traffic-value list. The new prediction is then appended and becomes part of the history used to generate the next step. This recursive strategy allows the application to create multi-hour forecasts from a single selected model.

Forecast Setting	Supported Values / Rule
Forecast horizon	6, 12, 24 or 48 hours
Minimum history	At least 30 observations for a selected junction
Non-negative demand	Predictions are clipped at zero
Predicted speed	Derived from configured base speed, capacity and forecast demand
Speed limits	Clipped to 12-65 km/h
Congestion	LOW < 0.65; MEDIUM 0.65-0.90; HIGH >= 0.90 demand/capacity ratio

5.5.1 Congestion Logic

For configured junctions, congestion is calculated from vehicles divided by the configured capacity. Ratios at or above 0.90 are labeled HIGH, ratios from 0.65 up to but below 0.90 are labeled MEDIUM, and lower ratios are labeled LOW. This rule is intentionally transparent so that the dashboard classification can be explained without treating the label as a learned model output.

5.5.2 Anomaly Detection

The application calculates the first and third quartiles of traffic volume independently for each junction, computes the interquartile range and flags observations below Q1 - 1.5×IQR or above Q3 + 1.5×IQR. The method is used as a monitoring aid rather than as a causal diagnosis.

5.6 Interfaces and Dashboard Design



Figure 5.5: User-interface flow used by the Streamlit application

The sidebar provides three main controls: traffic-data upload, forecast horizon and junction selection. The main panel presents a live-style dashboard view with current traffic, forecast peak, average forecast speed and current congestion. Supporting sections provide historical-versus-forecast charts, a speed chart, model feature importance, model metrics, an hourly forecast table, junction comparison, anomaly detection, traffic intelligence and downloadable results.

5.7 Data Schema and Processing Rules

Field	Type	Role in System
timestamp	Datetime	Primary time index for sorting and temporal features
junction	Text	Location identifier used for grouping and selection
vehicles	Numeric	Target variable for traffic-volume forecasting
speed	Numeric / optional	Historical speed; forecast speed is derived for configured junctions
temperature	Numeric / optional	Context feature; defaults to 25 if absent
rain	Numeric / optional	Context feature; defaults to 0 if absent
event	Numeric / optional	Event indicator; defaults to 0 if absent
hour, day_of_week, month, is_weekend	Derived	Calendar predictors
lag and rolling features	Derived	Historical demand predictors

6 Performance Test

6.1 Constraints and Test Objectives

The performance section is treated as an engineering validation step rather than only an academic model-score exercise. The goal is to determine whether the application behaves consistently under normal user actions, whether the forecasting pipeline produces valid outputs and whether the visible results align with the underlying rules.

Constraint / Quality Dimension	Why It Matters	How It Is Addressed
Prediction error	Forecast usefulness depends on accurate demand estimates	MAE, RMSE and R^2 on held-out rows
Speed response	Operational views should remain readable	Compact Plotly charts and cached model/data stages
Data quality	Bad timestamps or traffic values can break feature creation	Normalization, parsing, numeric coercion and invalid-row removal
History sufficiency	Lag-24 and rolling windows require enough records	Training drops rows with missing required features; forecasting requires >=30 observations
Congestion logic	Thresholds must be stable and interpretable	Fixed demand/capacity thresholds
Anomaly robustness	Single extreme values should not derail the workflow	IQR rule rather than mean-only thresholds
Export correctness	Users need usable outputs outside the dashboard	Separate TXT, CSV and Excel export paths

A production deployment would additionally need response-time measurements under realistic traffic-data volumes, live sensor availability checks, model drift monitoring and security testing. These are outside the current local prototype scope.

6.2 Test Plan / Test Cases

ID	Test Case	Expected Result	Observed / Code-Supported Result
TC-01	Load built-in demo dataset	Dataset is generated without upload	Pass; 120-day hourly data across 5 configured junctions
TC-02	Upload valid CSV	Required traffic/time fields are recognized	Pass; flexible timestamp/traffic column names supported
TC-03	Missing timestamp column	Application reports validation error	Pass; explicit error raised
TC-04	Missing traffic column	Application reports validation error	Pass; explicit error raised
TC-05	Select 6/12/24/48-hour horizon	Forecast length follows selection	Pass; four values exposed in sidebar
TC-06	Train model candidates	RF and GB models are trained	Pass; both fitted before model selection
TC-07	Select final model	Lower MAE model is used	Pass; MAE comparison drives selection
TC-08	Generate forecast	Predictions are numeric and non-negative	Pass; values are clipped at zero
TC-09	Classify congestion	Label follows demand/capacity rule	Pass; LOW/MEDIUM/HIGH thresholds implemented
TC-10	Detect anomalies	IQR outliers are identified	Pass; per-junction quartile rule implemented
TC-11	Export results	TXT/CSV/Excel downloads available	Pass; three download buttons implemented
TC-12	Compare junctions	Summary table is generated for available locations	Pass; current, peak and speed shown

6.3 Test Procedure

1. Start the Streamlit application and confirm that the dashboard loads with the built-in demonstration dataset.
2. Select a junction from the sidebar and choose the desired forecast horizon.
3. Record current traffic, forecast peak, average speed, congestion status and model metrics.
4. Inspect the historical-versus-forecast chart and the hourly forecast table for internal consistency.
5. Inspect feature importance to confirm that the fitted tree model exposes the expected predictors.
6. Review junction comparison, anomaly detection and traffic-intelligence messages.
7. Expand the raw-data view and check timestamp, junction and vehicle fields.
8. Download the text, CSV and Excel outputs and verify that each contains the expected forecast content.
9. Repeat the run with alternate forecast horizons or an uploaded CSV to verify the configurable input path.

Captured application run used in this report

The provided screenshots show Ayodhya Bypass selected with a 12-hour horizon. The dashboard displays the traffic forecast, speed graph, model feature importance, model performance, hourly forecast, junction comparison, anomaly monitoring, traffic intelligence and export controls.

6.4 Performance Outcome

The screenshots supplied for the final application provide one concrete demonstration run from the working dashboard. Because the displayed metrics are tied to that captured run, they should be interpreted as observed application output rather than as a claim of universal predictive accuracy across all traffic conditions.

Metric	Observed Value	Interpretation
Selected Model	Gradient Boosting Regressor	Lower-MAE candidate chosen by the application in the captured run
MAE	48.72 vehicles/hour	Average absolute deviation on the displayed test evaluation
RMSE	62.60 vehicles/hour	Penalizes larger errors more strongly than MAE
R^2	0.9662	High explained-variance value for the displayed evaluation split
Forecast horizon	12 hours	Selected in the captured dashboard
Selected junction	Ayodhya Bypass	Junction visible in the screenshots

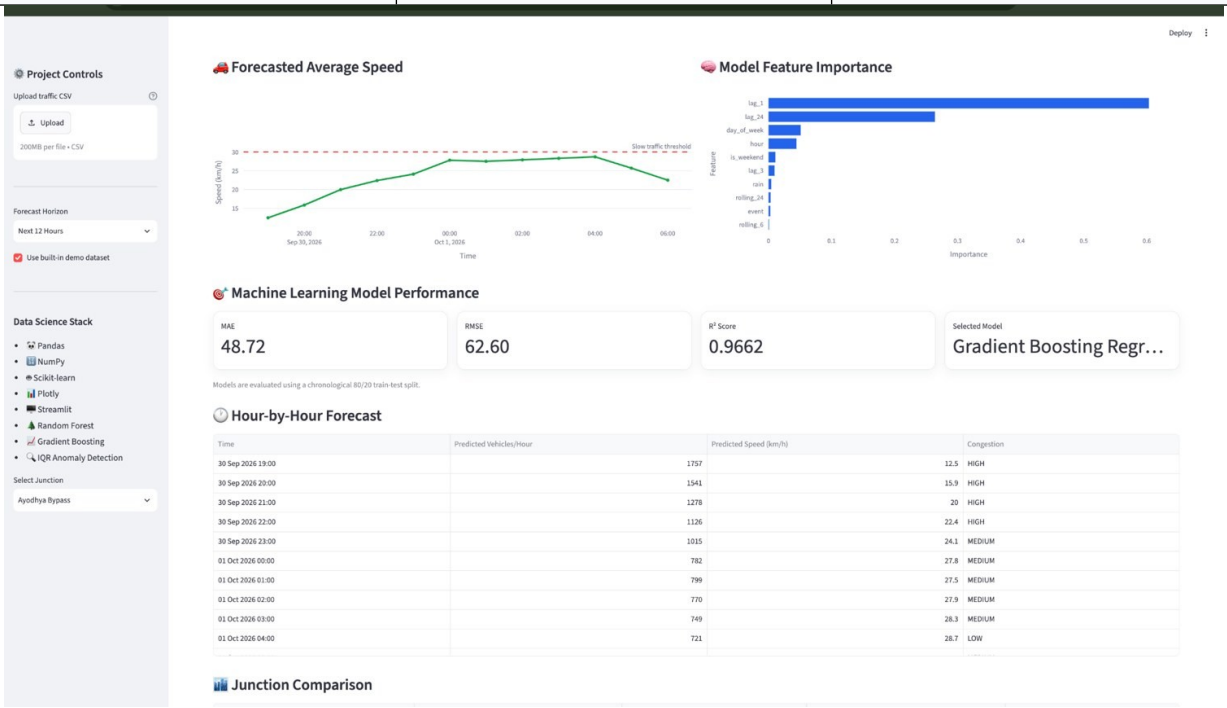


Figure 6.1: Model performance, forecast speed and feature-importance view from the working application

6.5 Observed Demonstration Results

Indicator	Captured Value	Comment
Current traffic	1,929 vehicles/hour	Latest displayed value for Ayodhya Bypass
Forecast peak	1,757 vehicles/hour	Displayed peak at 30 Sep 2026, 19:00
Average forecast speed	23.6 km/h	Below the dashboard's 30 km/h slow-traffic reference line
Current congestion	HIGH	Demand/capacity ratio exceeds the 0.90 HIGH threshold
Peak congestion	HIGH	Peak forecast also crosses the HIGH threshold
Average predicted demand	1,048 vehicles/hour	Average over the selected 12-hour horizon
Anomaly result	No significant anomalies detected	IQR method on the selected junction history

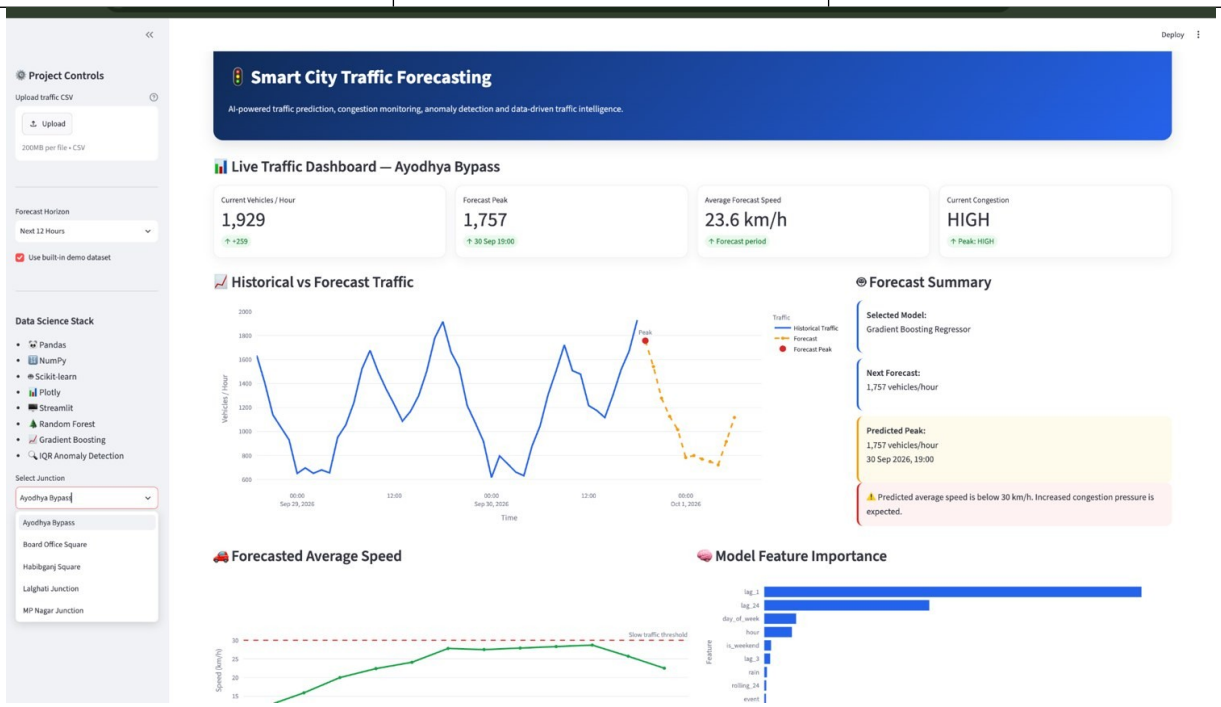


Figure 6.2: Overall dashboard view showing KPI cards, historical-vs-forecast traffic and selected junction

The screenshots further show the hourly forecast table, junction comparison table, anomaly-monitoring message, traffic-intelligence summary and export buttons. These outputs demonstrate that the project is not limited to a model score; it wraps the model in a complete decision-support interface.

7 My Learnings

7.1 Technical Learning

The project strengthened my practical understanding of how a machine-learning model is built as part of a complete software workflow. I learned that preprocessing and feature design are as important as algorithm selection because the model can only learn useful structure from well-formed predictors. The project also helped me understand why time-related variables, recent lag values and rolling statistics are valuable for traffic forecasting.

- Improved Python programming for data science using Pandas and NumPy.
- Hands-on experience with Scikit-learn regression models and evaluation metrics.
- Practical use of Random Forest and Gradient Boosting for nonlinear tabular prediction.
- Feature engineering for hourly and junction-wise traffic data.
- Interactive visualization with Plotly and dashboard development with Streamlit.
- Implementation of transparent rule-based congestion classification and IQR anomaly detection.
- Exporting analytical results to CSV and Excel for further use.

7.2 Analytical and Professional Learning

An important learning from the internship was the difference between building a model and building a usable solution. A model can produce predictions, but users also need context: when the peak is expected, how severe congestion may be, how fast traffic may move, which features matter and how the results can be exported. Designing the interface around these questions improved my ability to think from the user's perspective.

Problem-Solving Approach

- Break the project into smaller components: data, features, models, forecast logic, visualization and export.
- Use explicit validation rules for required data fields rather than assuming every input file is well formed.
- Compare models using a defined metric before selecting one for the final forecasting stage.
- Inspect both numeric metrics and visual outputs to catch issues that a single score may miss.
- Document limitations honestly, especially when demonstration data is synthetic or external inputs are not live.

Career Relevance

The experience is directly relevant to entry-level data-science and machine-learning work because it combines coding, statistics, predictive modeling, visualization, testing and communication. It also provides a concrete project that can be discussed in interviews from both a technical and an application perspective.

8 Future Work Scope

8.1 Planned Enhancements

- Integrate live traffic sensors, camera feeds or trusted traffic APIs so that the input layer continuously reflects the current city state.
- Add live weather, holiday and event data rather than reusing the most recent contextual values during recursive forecasting.
- Evaluate time-series-specific algorithms such as XGBoost-based feature systems, LSTM/GRU models, Prophet-style models or temporal deep-learning architectures where justified by the data volume and operating requirements.
- Add model persistence, automated retraining and model-version tracking so production deployments do not retrain from scratch on every application run.
- Introduce cross-validation strategies appropriate for time series, rolling-window evaluation and forecast calibration.
- Add map-based visualization so junction forecasts can be understood spatially across a city.
- Extend the decision layer to support adaptive signal-timing recommendations, diversion suggestions and alert notifications.
- Add authentication, role-based access and audit logging for production deployment.

8.2 Limitations of Current Implementation

Area	Current Limitation	Recommended Future Direction
Data source	Built-in dataset is synthetic; uploaded CSV is user supplied	Use calibrated historical sensor data and live streams
Future covariates	Temperature, rain and event use the latest available value in recursive forecasting	Fetch future exogenous variables or model scenarios
Location modeling	Junction is used for history grouping and capacity rules but is not encoded as a learned feature	Introduce junction embeddings/encoding or separate models where appropriate
Validation	Single 80/20 held-out split in the current implementation	Use rolling-origin / walk-forward evaluation
Model persistence	Models are trained when the app runs	Persist versioned models and metadata
Infrastructure	No direct signal-controller or city-system integration	Expose secure service/API interfaces
Monitoring	No automated drift monitoring or alert history store	Add model/traffic drift dashboards and retention

The future scope is deliberately focused on extensions that increase operational realism, reliability and maintainability while preserving the current modular design. The present application remains a functional prototype suitable for demonstration, learning and further development.

Appendix A Project Outcome Screenshots

The following screenshots were supplied by the author and document the working Streamlit application. They are included as evidence of the implemented user interface and observed outputs.

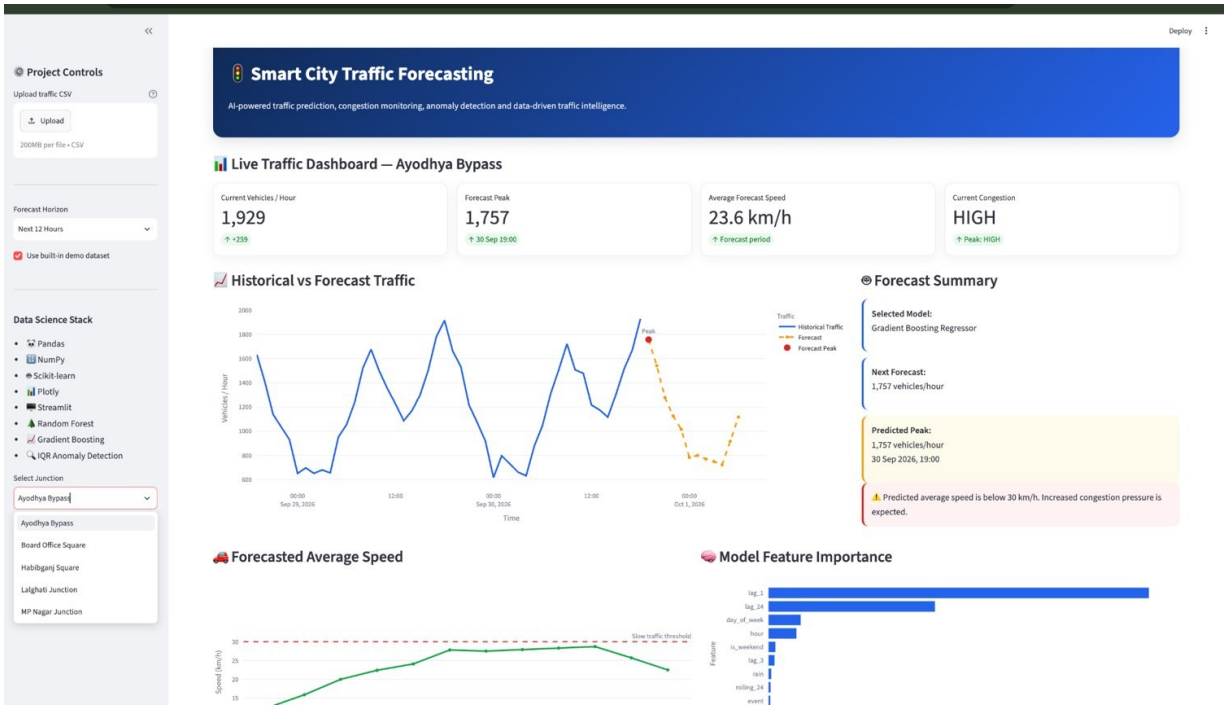


Figure A.1: Main dashboard with Ayodhya Bypass selected, KPI cards and historical-versus-forecast chart

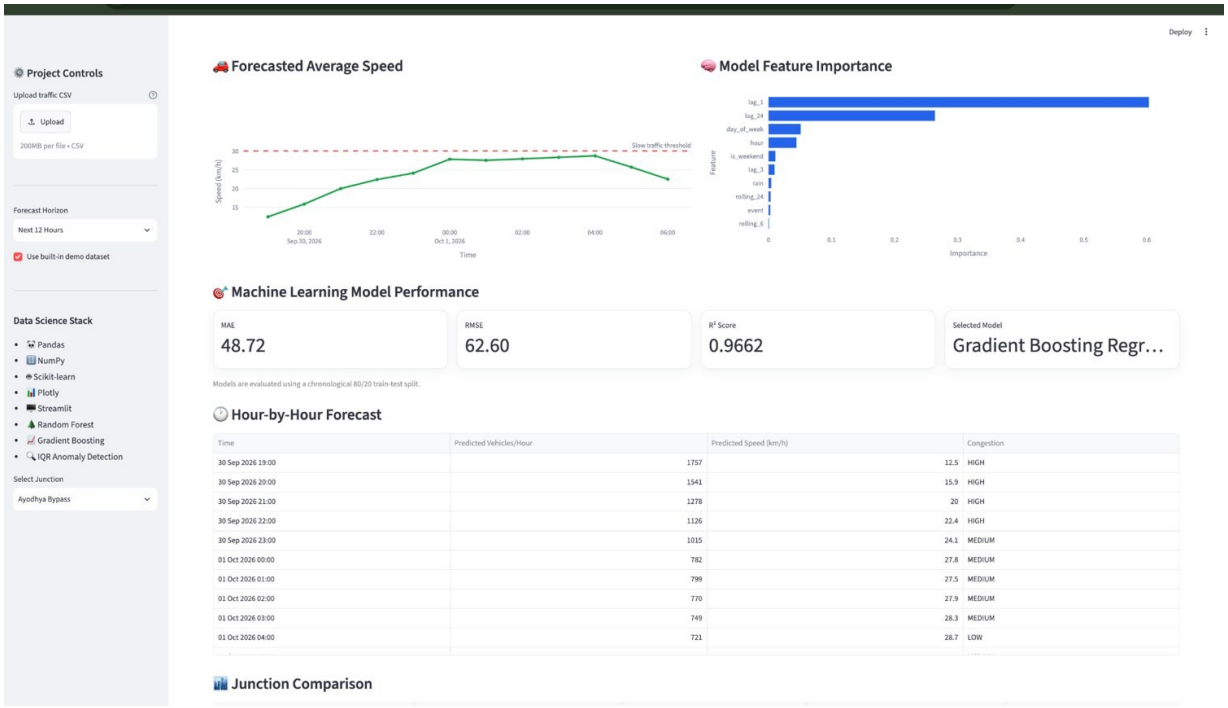


Figure A.2: Model performance, forecast speed, feature importance and hourly forecast table

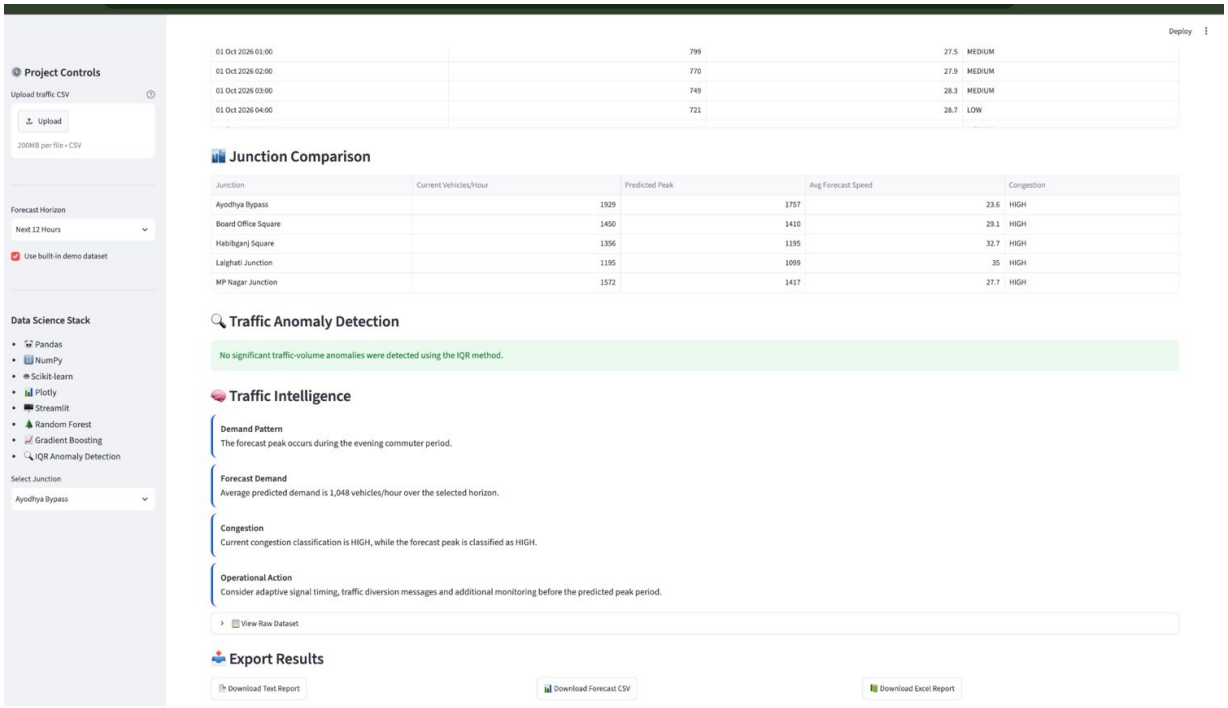


Figure A.3: Junction comparison, anomaly detection, traffic intelligence and export controls

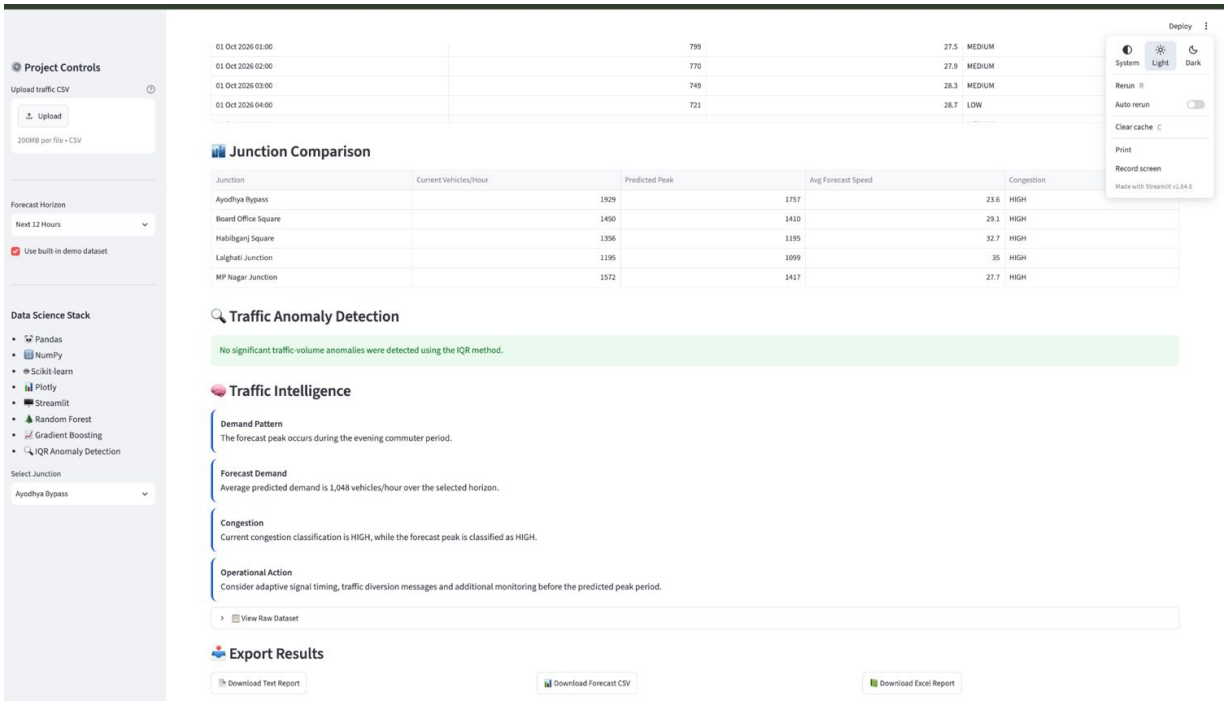


Figure A.4: Application menu view and lower dashboard sections

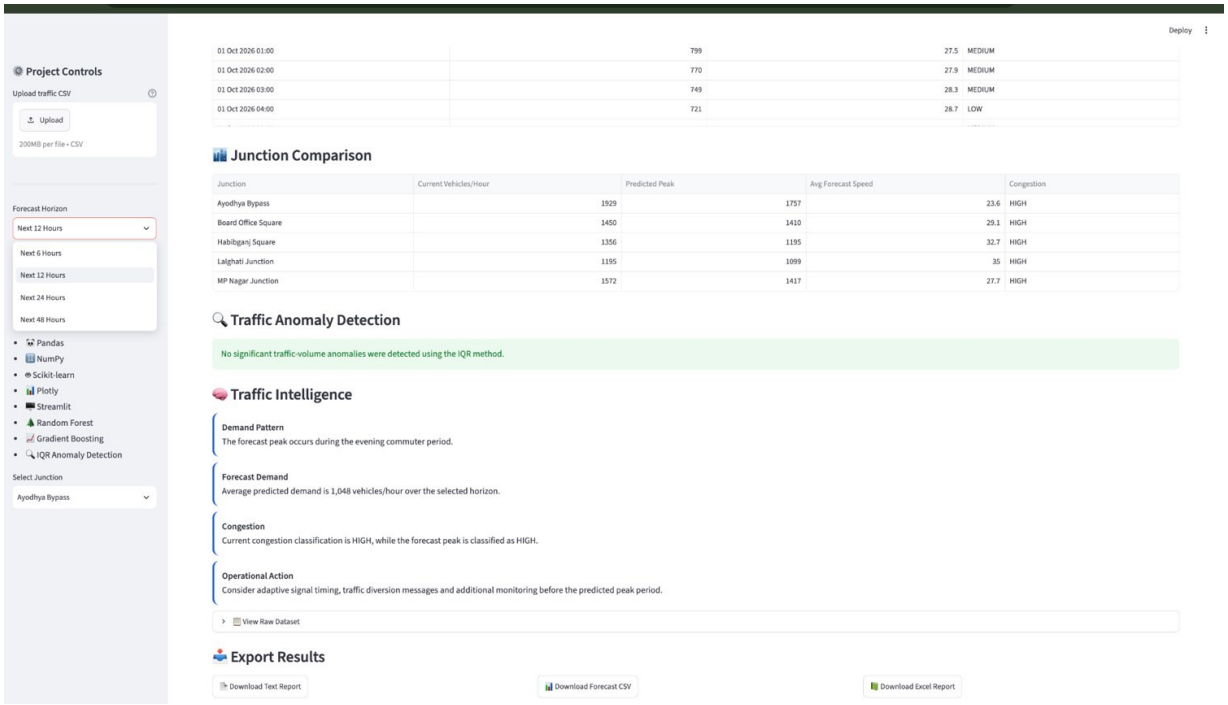


Figure A.5: Forecast-horizon selector showing 6, 12, 24 and 48-hour options

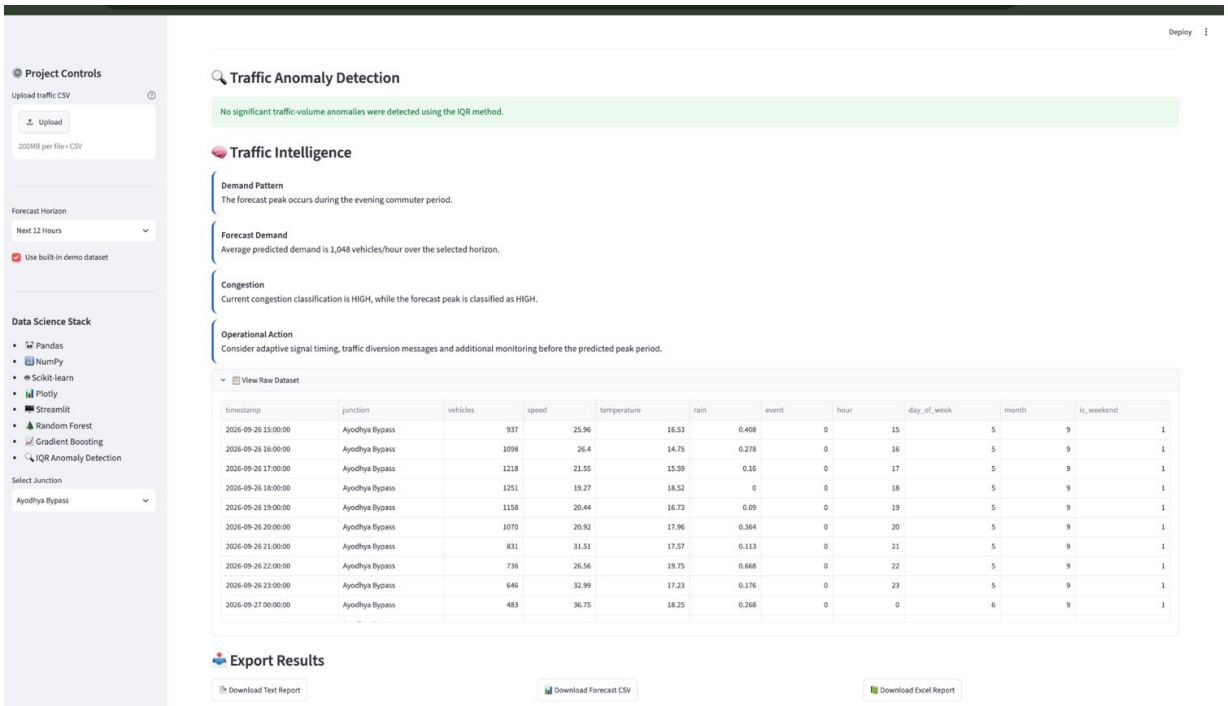


Figure A.6: Expanded raw-dataset view used to inspect the underlying traffic records

Appendix B Selected Code Components

The project source file contains the complete application. The excerpts below highlight the core implementation areas used to explain the architecture in this report.

B.1 Feature engineering

```
grouped = result.groupby("junction", group_keys=False)
result["lag_1"] = grouped["vehicles"].shift(1)
result["lag_2"] = grouped["vehicles"].shift(2)
result["lag_3"] = grouped["vehicles"].shift(3)
result["lag_24"] = grouped["vehicles"].shift(24)
result["rolling_3"] = grouped["vehicles"].shift(1).rolling(3).mean()
result["rolling_6"] = grouped["vehicles"].shift(1).rolling(6).mean()
result["rolling_24"] = grouped["vehicles"].shift(1).rolling(24).mean()
```

B.2 Model training and selection

```
rf = RandomForestRegressor(n_estimators=250, max_depth=18,
                           min_samples_leaf=2, random_state=42, n_jobs=-1)
gb = GradientBoostingRegressor(n_estimators=220, learning_rate=0.05,
                               max_depth=4, loss="huber", random_state=42)
rf.fit(X_train, y_train)
gb.fit(X_train, y_train)
rf_mae = mean_absolute_error(y_test, rf.predict(X_test))
gb_mae = mean_absolute_error(y_test, gb.predict(X_test))
model = rf if rf_mae <= gb_mae else gb
```

B.3 Forecast generation

```
for step in range(1, horizon + 1):
    timestamp = last_timestamp + pd.Timedelta(hours=step)
    lag_1 = values[-1]
    lag_2 = values[-2]
    lag_3 = values[-3]
    lag_24 = values[-24] if len(values) >= 24 else np.mean(values)
    rolling_3 = np.mean(values[-3:])
    rolling_6 = np.mean(values[-6:])
    rolling_24 = np.mean(values[-24:])
    prediction = float(model.predict(row[FEATURE_COLUMNS])[0])
    prediction = max(0, prediction)
    values.append(prediction)
```

B.4 Congestion and anomaly rules

```
ratio = vehicles / capacity
if ratio >= 0.90:
    return "HIGH"
if ratio >= 0.65:
    return "MEDIUM"
return "LOW"

q1 = group.quantile(0.25)
q3 = group.quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
```

B.5 Component-to-Function Mapping

Function	Responsibility
generate_dataset	Creates the built-in hourly demonstration dataset for the configured junctions.
prepare_uploaded_data	Normalizes uploaded CSV columns, parses timestamps and prepares required fields.
engineer_features	Builds temporal, lag and rolling predictors grouped by junction.
train_models	Trains Random Forest and Gradient Boosting, compares MAE and returns model metrics and feature importance.
make_forecast	Generates recursive multi-hour traffic forecasts and derives predicted speed.
congestion_label	Maps demand/capacity ratios to LOW, MEDIUM or HIGH.
detect_anomalies	Flags IQR-based outliers within each selected junction.
create_report	Builds the downloadable text report from the selected forecast and metrics.

These excerpts are condensed for documentation. The supplied source file remains the authoritative implementation.

Conclusion and References

CONCLUSION

The Smart City Traffic Forecasting project demonstrates an end-to-end data-science workflow for a practical urban-transportation problem. The completed application moves from traffic-data ingestion and validation to temporal feature engineering, model comparison, short-term prediction, congestion interpretation, anomaly detection, visualization and export. The final interface provides both technical outputs, such as MAE, RMSE, R^2 and feature importance, and user-oriented outputs, such as forecast peaks, average speed, congestion status and junction comparison.

The captured dashboard run confirms that the implemented solution functions as a usable prototype and produces visible model and forecasting outputs. The next stage is to connect the architecture to real traffic data, adopt stronger time-series validation and monitoring, and integrate the forecasting service with a broader smart-city operational platform.

REFERENCES

1. Scikit-learn User Guide and API Reference: RandomForestRegressor, GradientBoostingRegressor and model evaluation metrics.
2. Streamlit Documentation: building interactive Python data applications.
3. Plotly Python Documentation: interactive visualization and graph objects.
4. Pandas Documentation: data structures, time-series handling and data transformation.
5. NumPy Documentation: numerical arrays and vectorized computation.
6. Géron, A., Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow, O'Reilly.
7. James, G., Witten, D., Hastie, T., Tibshirani, R., An Introduction to Statistical Learning, Springer.
8. Project source code supplied by Harsh Kumar for the Smart City Traffic Forecasting internship project.